Whole Genome Sequencing and Variant Analysis of Mycobacterium tuberculosis

Author: Rishika Madhanagopal
Date: October 2026
Tools: FastQC, MultiQC, fastp, BWA-MEM, samtools, bcftools, SnpEff, SnpSift
Dataset source: https://doi.org/10.5281/zenodo.3497385

1. Project Overview:

In this project, I performed end-to-end variant analysis of a Mycobacterium tuberculosis whole-genome sequencing dataset. I started from raw FASTQ files and worked through quality control, adapter trimming, reference alignment, variant calling, quality filtering and functional annotation. My goal was to identify genetic variants in this TB strain and determine whether any are linked to antibiotic resistance, which has direct clinical relevance for treatment decisions.

The dataset contains paired-end Illumina reads at 251 bp read length, along with the *M. tuberculosis* H37Rv reference genome (NCBI accession GCF_000195955.2, 4,411,532 bp).

Workflow:

Raw FASTQ → QC → Trimming → Alignment → BAM Processing → Coverage → Variant Calling → Filtering → Annotation → Interpretation

2. Environment Setup

I created a dedicated Conda environment with all required tools to keep dependencies isolated and the analysis reproducible:

Bash:

```
conda create -n ngs -c conda-forge -c bioconda \
    fastqc multiqc fastp bwa samtools bcftools snpeff
conda activate ngs

mkdir NGS_MTB && cd NGS_MTB
mkdir Data Scripts Qc Trimmed Alignment Variant Annotation Figures Results
```

I placed TB_R1.fastq, TB_R2.fastq and the reference FASTA in the Data/ folder.

3. Quality Control

Tools: FastQC, MultiQC

Before doing anything to the data, I ran FastQC on both raw read files. I did this first because QC results directly inform every trimming decision I would make next. Going straight to trimming without checking the data would mean trimming blindly.

Bash:

```
fastqc Data/TB_R1.fastq Data/TB_R2.fastq -o Qc/fastqc_raw
multiqc Qc/fastqc_raw/ -o Qc/multiqc_raw
```

What I found:

Metric	R1	R2
Total reads	675,157	675,157
Read length	251 bp	251 bp
%GC	63%	59%
Mean quality per read	Q37	Q27
Adapter contamination	Nextera ~60%	Nextera ~44%

Per base sequence quality: Both R1 and R2 showed quality dropping toward the 3' end of reads. This is normal for Illumina chemistry as the fluorescent signal degrades over successive sequencing cycles. R2 was noticeably worse than R1, with a mean quality of only Q27 compared to Q37 in R1. This makes sense because R2 is sequenced after R1 on the same clusters, by which point the signal has deteriorated further. This difference between R1 and R2 told me I would need quality trimming, especially for R2.

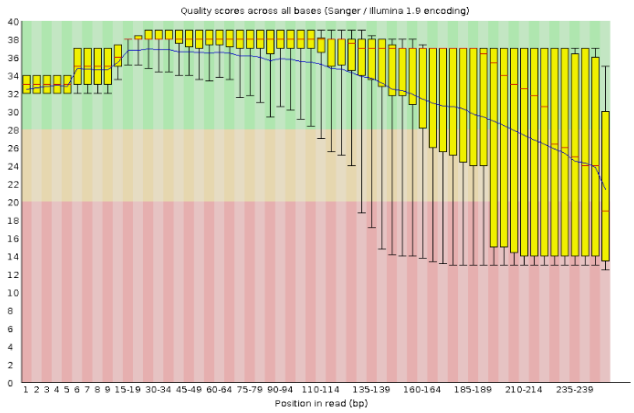


Figure 1: R1 per-base quality (raw) 3′ decline into warning zone after position 100.

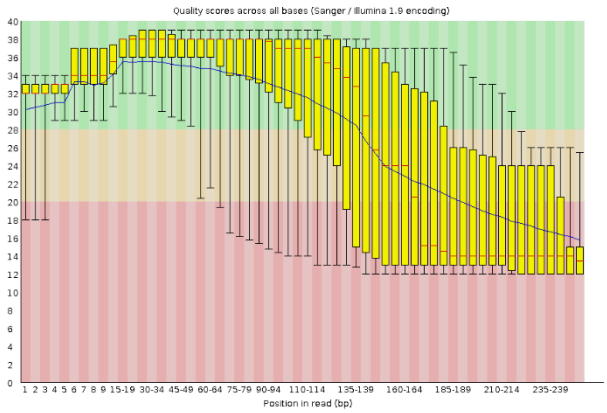


Figure 2: R2 per-base quality (raw) severe 3′ decline below Q20 after position 100.

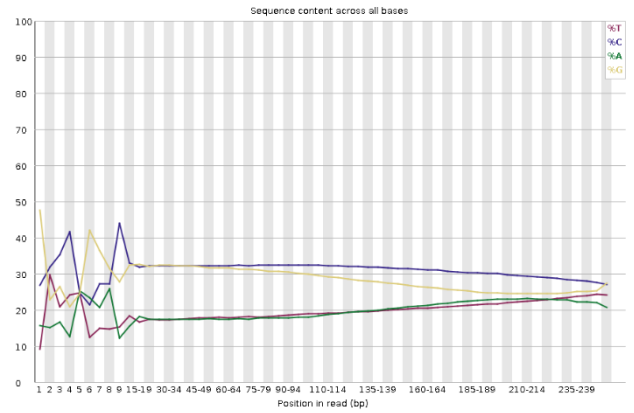


Figure 3: R1 per-base sequence content (raw) early positional bias in the first 15 bp.

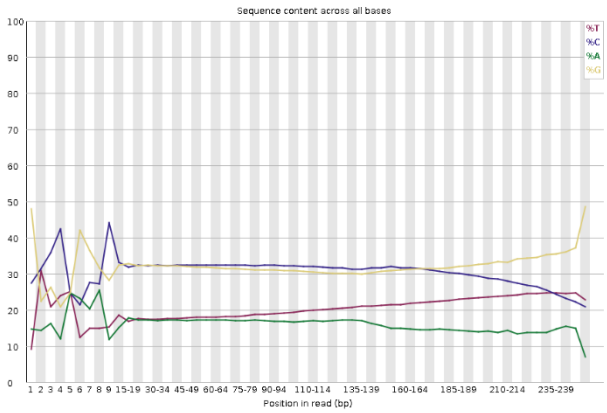


Figure 4: R2 per-base sequence content (raw) early positional bias with 3′ GC divergence.

Adapter contamination: The adapter content plot showed Nextera transposase sequences rising steeply from position 100 onwards, reaching 60% in R1 and 44% in R2 by the end of the reads. I found this alarming at first, but it makes sense once you consider the insert size. fastp later confirmed the library insert size was 130 bp. Since the read length is 251 bp but most fragments are only 130 bp, both R1 and R2 read through the entire fragment and into the adapter at the other end. This is called adapter read-through, and it means adapter removal was essential before alignment.

G spike in R2: The per-base sequence content plot showed a sharp spike in G content at the very last positions of R2. This is another signature of adapter read-through. Illumina sequencers call uncycled positions as G by default, and Nextera adapters are G-rich. This confirmed the adapter problem from a different angle.

GC content: R1 showed a GC peak at 63%, and R2 at 59%, both consistent with the known M. tuberculosis GC content of ~65%. After trimming, GC shifted to 65.1% as Nextera adapter sequences (which are lower GC) were removed.

Tile quality: I noticed below-average quality in tiles 2110-2114. This is a physical artefact from the flow cell, probably a bubble or obstruction during the run. I decided not to filter by tile because this type of issue affects only a small fraction of the data, and standard TB genomics pipelines do not remove tile-based reads unless coverage is critically low.

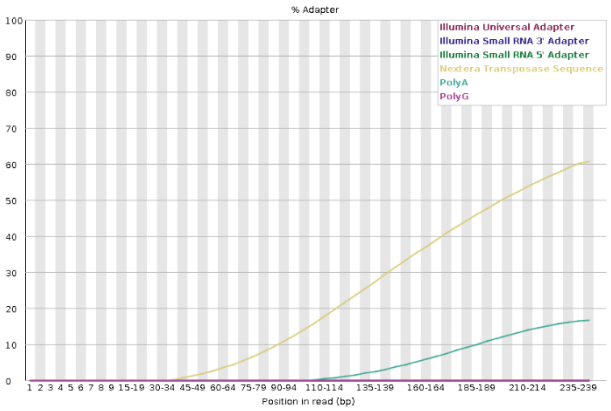


Figure 5: R1 adapter content before trimming: Nextera contamination reaches ~60% at the 3′ end.

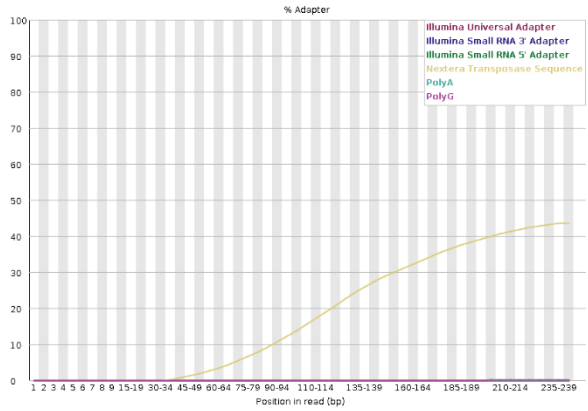


Figure 6: R2 adapter content before trimming: Nextera contamination reaches ~45% at the 3′ end.

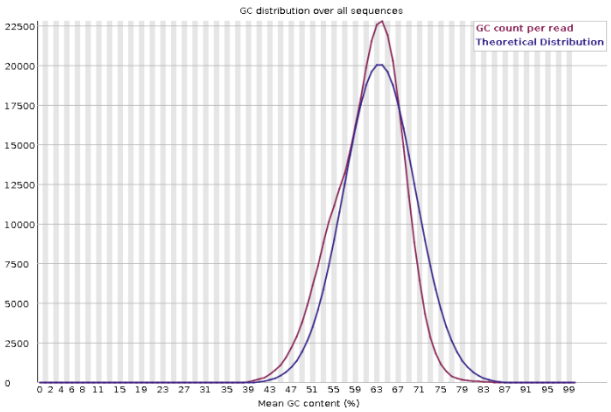


Figure 7: R1 GC content before trimming: slight deviation from theoretical distribution.

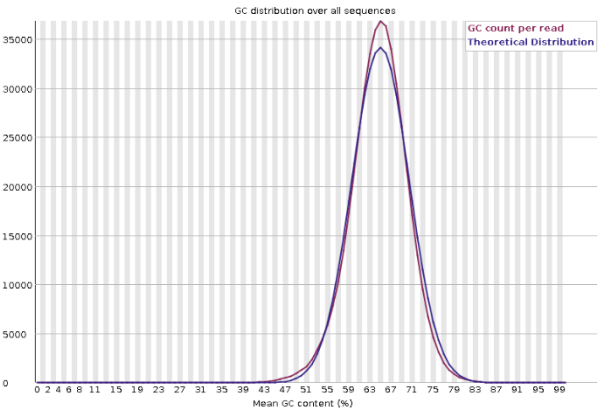


Figure 8: R1 GC content after trimming; observed distribution closely follows the theoretical curve

4. Trimming

Tool: fastp v1.3.7

Based on what I saw in QC, I needed to remove Nextera adapter sequences, trim low-quality bases from the 3' end of reads (especially R2), and discard reads that become too short to align reliably. I chose fastp because it handles all three in a single step and auto-detects adapter sequences directly from the data rather than requiring me to specify them manually.

Bash:

```
fastp \
  -i Data/TB_R1.fastq \
  -I Data/TB_R2.fastq \
  -o Trimmed/trimmed_R1.fastq \
  -O Trimmed/trimmed_R2.fastq \
  -h Results/trim_fastp.html \
  -j Results/trim_fastp.json \
  --detect_adapter_for_pe \
  --qualified_quality_phred 20 \
  --cut_tail \
  --cut_tail_mean_quality 20 \
  --length_required 50
```

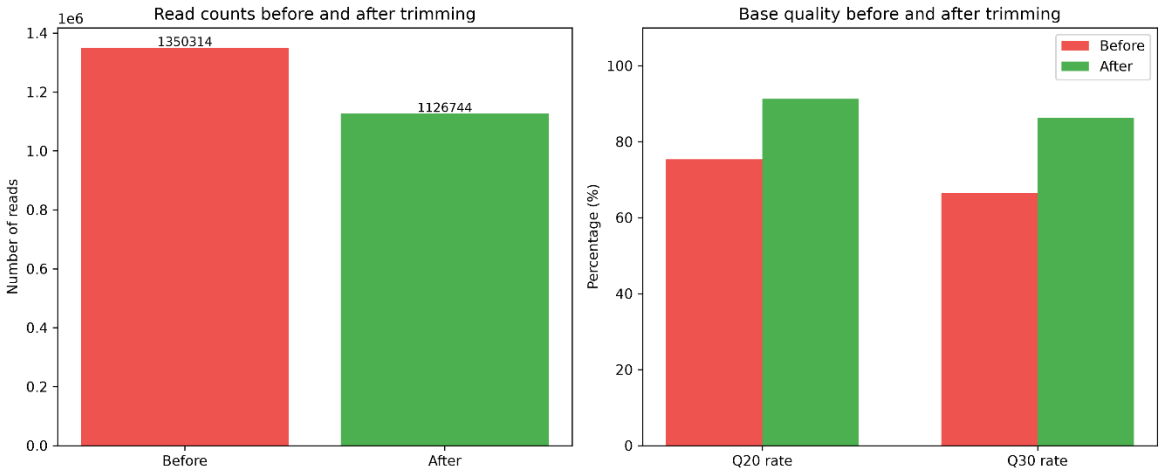


Figure 9: fastp trimming summary showing read counts and bases removed before and after adapter clipping.

Why I chose each parameter:

I used **--detect_adapter_for_pe** so fastp would identify adapters by analysing overlap between R1 and R2 reads rather than guessing from a preset list. It correctly identified both Nextera sequences: CTGTCTCTTATACACATCTCCGAGCCCACGAGAC (R1) and CTGTCTCTTATACACATCTGACGCTGCCGACGA (R2).

I set **--qualified_quality_phred 20** because Q20 represents 99% base-call accuracy, which is the accepted minimum for variant calling. Below this, errors are common enough to be confused with real variants. I had to set this explicitly because raw R2 was only 54.7% Q30, meaning a large portion of bases were below adequate quality.

I enabled **--cut_tail** and **--cut_tail_mean_quality 20** to trim low-quality tails from the 3' end. The sliding window moves from the end of the read toward the start and cuts when the window average drops below Q20. This directly addressed the quality drop I saw in the per-base quality plots, particularly in R2 after position 110.

I set **--length_required 50** because reads shorter than 50 bp after trimming tend to map to multiple genome locations by chance, producing unreliable alignments. Discarding them is safer than keeping them and getting false alignment signals.

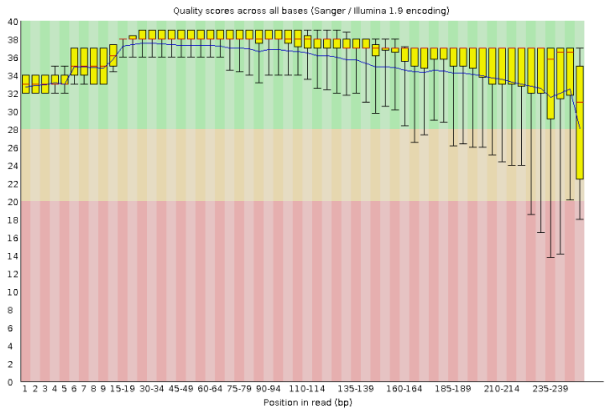


Figure 10: Per-base sequence quality after trimming for R1: reads maintain Q30+ across all 239 positions.

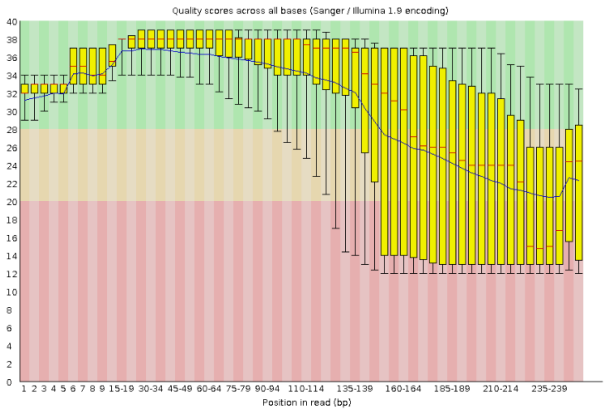


Figure 11: Per-base sequence quality after trimming for R2: quality declines toward the 3′ end, dropping into the warning zone after position 135.

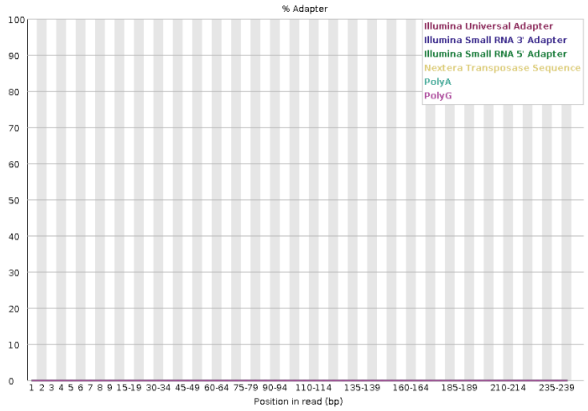


Figure 12: Adapter content after trimming for R1: no residual adapter sequences detected across all positions.

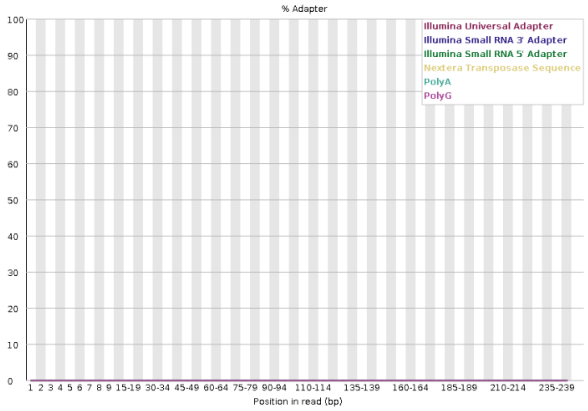


Figure 13: Adapter content after trimming for R2: no residual adapter sequences detected across all positions.

Results:

Metric	Before	After
Total read pairs	675,157	563,372 (83.4% retained)
Reads removed (low quality)	-	199,704
Reads removed (too short)	-	23,866
Mean read length R1	251 bp	179 bp
Mean read length R2	251 bp	172 bp
Overall Q20 rate	75.4%	91.3%
Overall Q30 rate	66.5%	86.3%
R1 Q30 rate	78.4%	93.6%
R2 Q30 rate	54.7%	78.7%
GC content	61.5%	65.1%
Adapter contamination	Yes (Nextera)	Fully removed

I was pleased with the improvement in R2 quality, which jumped from 54.7% to 78.7% Q30. The 7 bp difference in mean length between R1 (179 bp) and R2 (172 bp) shows the tail trimming cut more from R2, which had worse 3' quality. GC content shifted from 61.5% to 65.1% because the Nextera adapters (which have lower GC) were removed, shifting the composition closer to the true TB genome GC of ~65%.

I noticed the insert size estimate changed from 130 bp (first trim-Default) to 251 bp (after retrimming-Threshold). I investigated this and concluded the 130 bp figure is the accurate

library insert size. The shift to 251 bp in the retrim happened because aggressive quality trimming reduced read overlap detectability, which the insert size calculation relies on.

I ran FastQC and MultiQC again on the trimmed files to confirm the result:

Bash:

```
fastqc Trimmed/trimmed_R1.fastq Trimmed/trimmed_R2.fastq -o
Qc/fastqc_trimmed
multiqc Qc/fastqc_trimmed/ -o Qc/multiqc_trimmed
```

Post-trim FastQC confirmed adapter content was flat at zero for both files. R1 per-base quality earned a green pass. The remaining flags (tile quality, early position base composition wobble) are artefacts of library preparation and the sequencing run that trimming cannot fix, and they are not expected to affect alignment or variant calling.

5. Alignment

Tool: BWA-MEM v0.7.19

I chose BWA-MEM as my aligner because it is the standard tool for aligning Illumina short reads to bacterial reference genomes and handles variable read lengths well, which mattered here since my reads now ranged from 50-251 bp after trimming.

First, I indexed the reference genome. This builds a data structure that lets BWA jump directly to relevant positions rather than scanning the full 4.4 Mb for every read:

Bash:

```
bwa index
Data/Mycobacterium_tuberculosis_h37rv.ASM19595v2.dna.chromosome.Chromosome.
fasta
```

Then I aligned my trimmed reads:

Bash:

```
bwa mem \

Data/Mycobacterium_tuberculosis_h37rv.ASM19595v2.dna.chromosome.Chromosome.
fasta \
  Trimmed/trimmed_R1.fastq \
  Trimmed/trimmed_R2.fastq \
  -o Alignment/alignment.sam
```

6. BAM Processing

Tool: samtools v1.24

I processed the SAM file through several steps before variant calling. Each step had a specific reason.

Bash:

```
# Compress to BAM
samtools view -b Alignment/alignment.sam -o Alignment/alignment.bam

# Sort by name so fixmate can find read pairs
samtools sort -n Alignment/alignment.bam -o
Alignment/alignment.namesorted.bam
```

```
# Add mate score tags needed by markdup
samtools fixmate -m Alignment/alignment.namesorted.bam
Alignment/alignment.fixmate.bam

# Sort by coordinate for markdup and variant calling
samtools sort Alignment/alignment.fixmate.bam -o
Alignment/alignment.sorted.bam

# Mark PCR duplicates
samtools markdup Alignment/alignment.sorted.bam
Alignment/alignment.markdup.bam

# Index the final BAM
samtools index Alignment/alignment.markdup.bam

# Get alignment statistics
samtools flagstat Alignment/alignment.markdup.bam
samtools stats Alignment/alignment.markdup.bam >
Alignment/samtools_stats.txt
```

I marked duplicates because during library preparation, DNA gets amplified by PCR. Sometimes the same original fragment gets copied many times, and multiple copies end up sequenced. These are PCR duplicates. If I left them in, they would artificially inflate my confidence in variant calls. If 8 out of 10 reads supporting a variant are duplicates of the same original fragment, I really only have 2 independent pieces of evidence, not 10. I marked them rather than deleted them so I still have a complete record, but variant callers will ignore marked duplicates during pileup.

Alignment statistics:

Metric	**Value**
Total reads	1,133,867
Mapped reads	1,129,218 (99.59%)
Properly paired	1,115,140 (98.97%)
Duplicates marked	5,474 (0.49%)
Singletons	125 (0.01%)
Mean base quality	Q34.3
Error rate	0.6%
Mean read length	176 bp
Insert size mean	209.4 bp

The 209.4 bp insert size from samtools stats reflects the mean after quality trimming shortened reads; the true library insert size of 130 bp was estimated from the initial fastp run before aggressive trimming reduced read overlap.

I was very satisfied with the 99.59% mapping rate. This confirmed the sample is genuinely *M. tuberculosis* and that I used the correct reference. A low mapping rate would have suggested contamination or a reference mismatch. The 98.97% properly paired rate is equally strong, meaning R1 and R2 of nearly every pair both mapped correctly in the expected orientation.

The 0.49% duplicate rate is low, confirming efficient library preparation with minimal PCR over-amplification.

7. Coverage

Tools: samtools depth, awk

Bash:

```
samtools depth -a Alignment/alignment.markdup.bam > Alignment/coverage.txt
awk '{sum+=$3} END {print "Mean coverage:", sum/NR}' Alignment/coverage.txt
```

I used -a to include every genome position in the output, including zeros. Without it, it skips uncovered positions, which would give a falsely high mean.

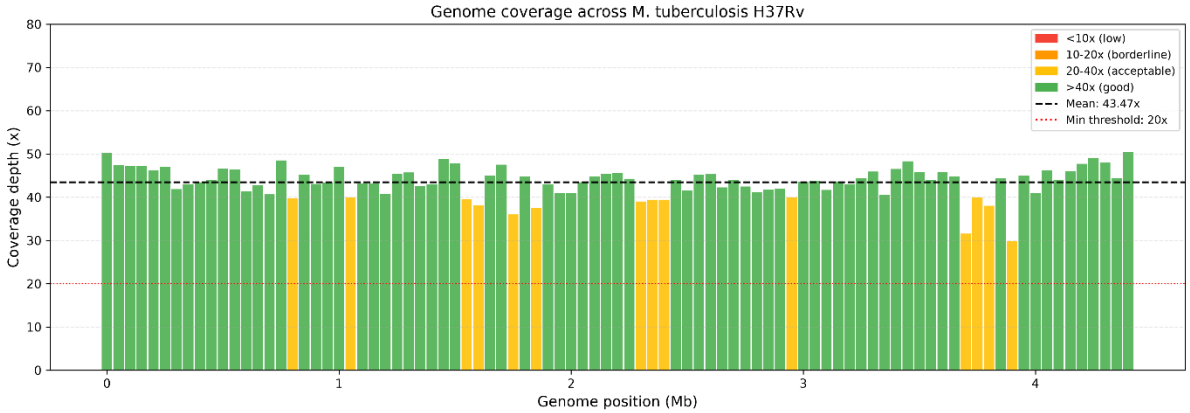


Figure 14: Per-chromosome coverage depth across the H37Rv reference genome, colour-coded by coverage tier.

Coverage results:

Metric	Value
Mean coverage	43.47x
Min	0x
Max	188x
Genome at ≥20x	93%

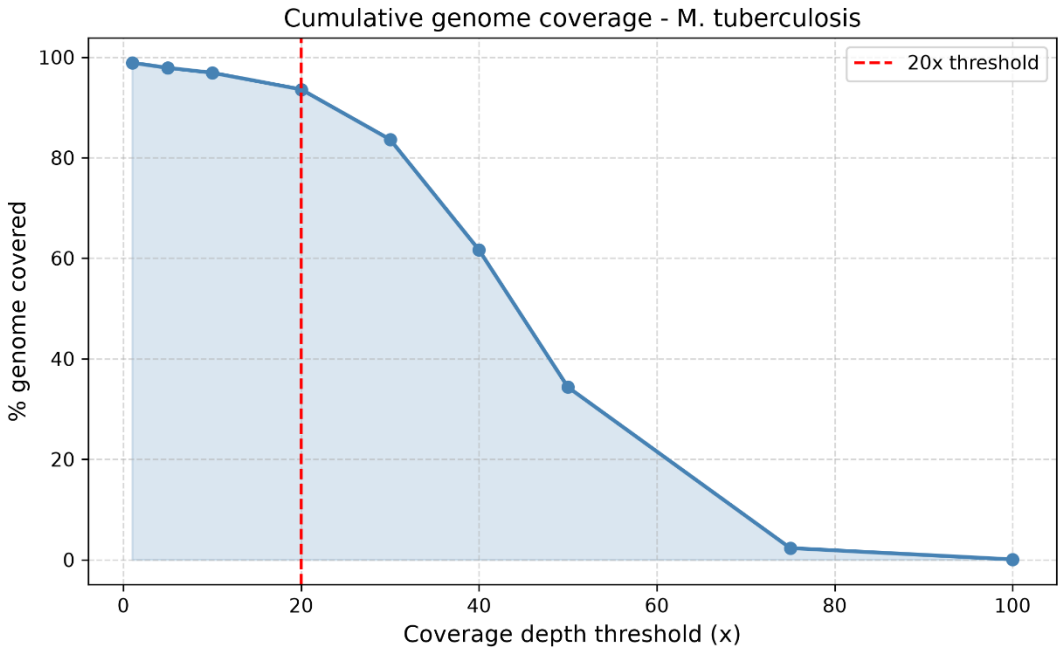


Figure 15: Cumulative coverage curve showing 93% of the genome covered at ≥20× depth.

I was satisfied with 43.47x mean coverage, which is well above the 20x minimum threshold I needed for confident variant calling. 93% of the TB genome was covered at 20x or above. The

7% with lower or zero coverage corresponds to repetitive regions, particularly PE/PPE gene families, where reads from multiple similar genome locations cannot be assigned to one specific position and simply do not map.

8. Variant Calling

Tool: bcftools v1.24

```
Bash:

bcftools mpileup \
  -f
Data/Mycobacterium_tuberculosis_h37rv.ASM19595v2.dna.chromosome.Chromosome.
fasta \
  Alignment/alignment.markdup.bam \
  -o Variant/alignment.mpileup.bcf -O b

bcftools call -m -v \
  --ploidy 1 \
  -o Variant/variant.raw.vcf \
  Variant/alignment.mpileup.bcf
```

I set --ploidy 1 because *M. tuberculosis* is haploid. It has one chromosome with one copy of every gene. If I had left the default diploid setting, bcftools would try to call heterozygous variants that simply cannot exist in this organism, which would produce biologically meaningless results. Setting ploidy to 1 ensures the variant caller makes the correct assumptions about allele counts.

Raw variant counts:

Type	Count
Total	1,180
SNPs	1,082
INDELs	98

9. Variant Filtering

Tool: bcftools filter

Before setting thresholds, I examined the actual quality and depth distributions in my data to choose thresholds that made sense for this specific dataset.

```
Bash:

# Quality distribution
grep -v "^#" Variant/variant.raw.vcf | awk '{print $6}' | sort -n | \
  awk '{if($1<20) low++; else high++} \
  END {print "Low quality (<20):", low, "\nHigh quality (>=20):", high}'

# Depth distribution
grep -v "^#" Variant/variant.raw.vcf | grep -o "DP=[0-9]*" | \
  cut -d= -f2 | sort -n | uniq -c | head -30
```

What I found:

- 54 variants (4.6%) had QUAL below 20, with some as low as 3.2
- 124 variants (10.5%) had depth below 10x
- 83 of those low-depth variants were supported by only 1-2 reads

Based on this, I chose QUAL≥20 and DP≥10 as my thresholds.

I chose QUAL≥20 because it represents 99% confidence per call. The variants below Q20 had quality scores suggesting roughly a 50% or higher probability of being false positives. There is no point in carrying them forward to annotation.

I chose DP≥10 because with my mean coverage of 43.47x, a variant site with only 1-2 supporting reads is almost certainly in a poorly mapped region. A single read difference from the reference could be a sequencing error or a misaligned read from a repetitive region. Requiring at least 10 independent reads gives enough evidence to distinguish a real variant from noise, while still being conservative enough that I am not throwing away good data.

Bash:

```
bcftools filter \
  -e 'QUAL<20 || DP<10' \
  -s LOWQUAL \
  Variant/variant.raw.vcf \
  -o Variant/variant.filtered.vcf

bcftools view -f PASS \
  Variant/variant.filtered.vcf \
  -o Variant/variant.pass.vcf

sed 's/^Chromosome/AL123456.3/' \
  Variant/variant.pass.vcf > Variant/variant.pass.renamed.vcf
```

I renamed the chromosome from Chromosome to AL123456.3 because bcftools used the name from my reference FASTA header, but the SnpEff database I built uses the NCBI accession. Without this rename, SnpEff would find no matching positions and produce empty annotations.

Filtering results:

Type	Raw	Filtered	Removed
Total	1,180	1,133	47
SNPs	1,082	1,035	47
INDELs	98	98	0

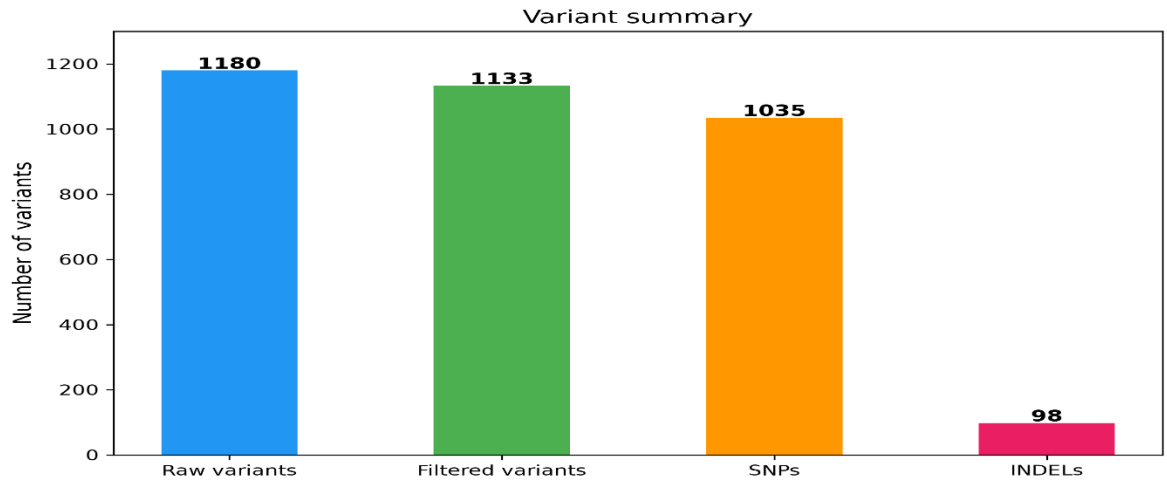


Figure 16: Raw and filtered SNP and INDEL counts after variant calling.

I found it interesting that all 47 removed variants were SNPs and all 98 INDELs passed. Removing only 3.98% of raw calls confirmed my thresholds were reasonable, not overly strict.

10. Annotation

Tool: SnpEff v5.4c, SnpSift

I used SnpEff to add biological meaning to my variant coordinates. Knowing a variant exists at position 761,109 is not useful on its own. I needed to know what gene it falls in, whether it changes the protein, and what the functional consequence might be.

Building a custom database:

I first checked whether a pre-built SnpEff database existed for my TB reference:

```
Bash:

snpEff databases | grep "Mycobacterium_tuberculosis_h37rv"
```

A database named Mycobacterium_tuberculosis_h37rv was available, but I built a custom one from the NCBI GenBank annotation file (GCA_000195955.2) to ensure the chromosome naming was exactly compatible with my renamed VCF file.

```
Bash:

mkdir -p ~/NGS/snpeff_data/TB_h37rv

cp
Data/Mycobacterium_tuberculosis_h37rv.ASM19595v2.dna.chromosome.Chromosome.
fasta \
   ~/NGS/snpeff_data/TB_h37rv/sequences.fasta

cp Data/ncbi_dataset/data/GCA_000195955.2/genomic.gbff \
   ~/NGS/snpeff_data/TB_h37rv/genes.gbk

nano $CONDA_PREFIX/share/snpeff-5.4.0c-0/snpEff.config
```

I added these two lines to the config (replacing YOUR_USERNAME with my actual username):

data.dir = /home/YOUR_USERNAME/NGS/snpeff_data/
TB_h37rv.genome: Mycobacterium tuberculosis H37Rv

```
Bash:

snpEff build -genbank -v TB_h37rv
```

Running annotation:

```
Bash:

snpEff -v TB_h37rv \
  Variant/variant.pass.renamed.vcf \
  > Annotation/variant.annotated.vcf

mv snpEff_summary.html Results/
mv snpEff_genes.txt Results/
```

Extracting results into a readable table:

```
Bash:

SnpSift extractFields \
  Annotation/variant.annotated.vcf \
  CHROM POS REF ALT QUAL \
  "ANN[0].GENE" \
```

```
  "ANN[0].EFFECT" \
  "ANN[0].IMPACT" \
  "ANN[0].HGVS_C" \
  "ANN[0].HGVS_P" \
  > Results/variants_table.txt

awk -F'\t' 'NR==1 || $8=="HIGH" || $8=="MODERATE"' \
  Results/variants_table.txt > Results/key_variants_table.txt
```

11. Results and Interpretation

11.1 Annotation summary

SnpEff annotated all 1,133 variants, generating 11,201 annotations in total. The higher annotation count reflects that each variant can affect multiple nearby genes, so SnpEff reports an annotation for each one. The variant rate of one per 3,893 bases tells me this strain has moderate genomic divergence from the H37Rv laboratory reference.

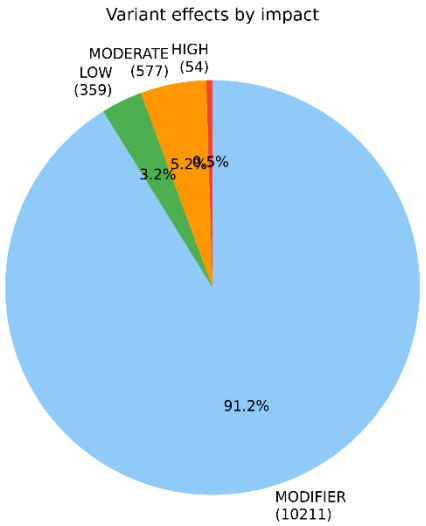


Figure 17: SnpEff functional impact classification of all annotated variants (n = 11,201).

By impact:

Impact	Count	%	What it means
HIGH	54	0.48%	Frameshift, stop codon, likely destroys function
MODERATE	577	5.15%	Missense, amino acid change, may affect function
LOW	359	3.21%	Synonymous, protein sequence unchanged
MODIFIER	10,211	91.16%	Intergenic, upstream, downstream

Note: The following counts reflect SnpEff annotations (not distinct variants, as one variant can annotate multiple genes)

By functional class:

Class	Count	%
Missense	552	60.1%
Synonymous	359	39.1%
Nonsense	7	0.76%

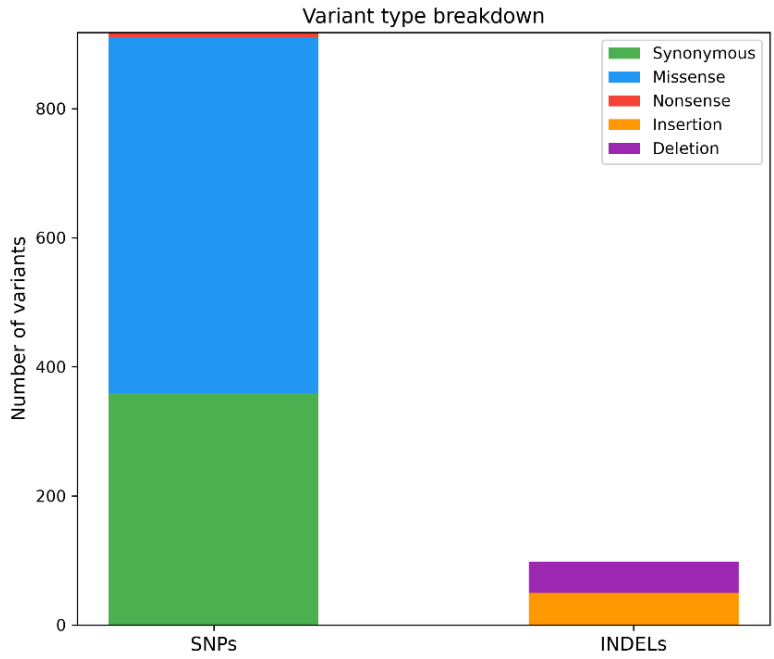


Figure 18: Variant type breakdown showing SNP subtypes (synonymous, missense, nonsense) and INDEL subtypes (insertion, deletion).

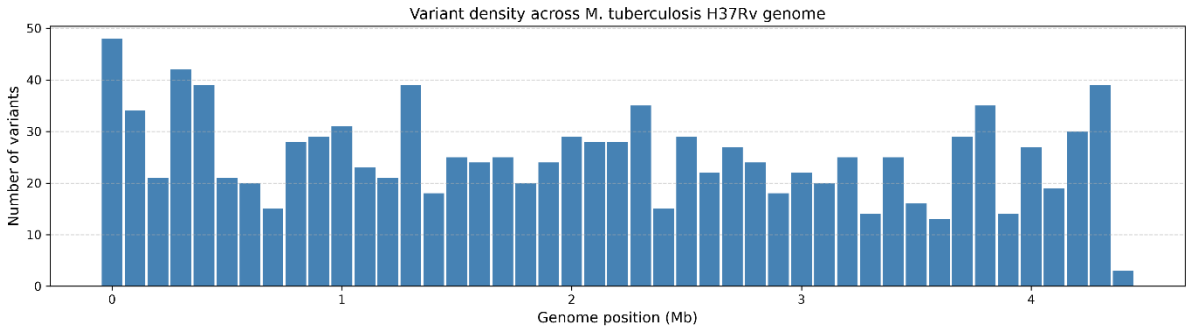


Figure 19: Variant density across the *M. tuberculosis* H37Rv genome in 100 kb windows.

I found the missense-to-silent ratio of 1.54 notable. Under neutral evolution, this ratio is typically close to 1. A higher ratio suggests some of the amino acid-changing variants may be under positive selection, meaning they provide a fitness advantage to the bacterium and have been retained.

The Ts/Tv ratio of 1.67 fell within the expected range of 1.5-2.0 for bacterial WGS. This reassured me that my variant calls are not enriched with sequencing errors, which tend to produce transversions and would push this ratio below 1.5.

11.2 HIGH impact variants

I found 51 variants causing complete loss of gene function. These include 43 frameshifts, 8 premature stop codons and 1 stop-loss event. The ones I considered most biologically interesting were:

Gene	Effect	Change	What the gene does
sigM (Rv3911)	frameshift	p.Arg160fs	Sigma factor controlling stress response
lpdA (Rv3303c)	stop gained	p.Cys472*	Central carbon metabolism enzyme
PE35 (Rv3872)	stop gained	p.Glu99*	Virulence and immune evasion
ppsA (Rv2931)	frameshift	p.Gln808fs	Cell wall lipid biosynthesis
gmhA (Rv0113)	frameshift	p.Phe179fs	Lipopolysaccharide biosynthesis

The disruption of sigM concerned me most among these because sigma factors control gene expression for entire sets of genes. Disrupting sigM could have wide downstream effects on how this bacterium responds to stress. The frameshift in ppsA, which is involved in building the TB cell wall lipid layer associated with immune evasion, also stood out as potentially affecting how this strain interacts with the host immune system.

11.3 Drug resistance mutations

I searched my annotated VCF specifically for known drug resistance genes:

```
Bash:

grep -E "rpoB | katG | gyrA | embB" Results/variants_table.txt
```

I found six clinically significant mutations across four genes:

Position	Gene	Nucleotide	Amino Acid	Drug
761,109	rpoB	c.1303G>T	p.Asp435Tyr	Rifampicin
2,155,168	katG	c.944G>C	p.Ser315Thr	Isoniazid
4,247,729	embB	c.1216G>A	p.Gly406Ser	Ethambutol
7,362	gyrA	c.61G>C	p.Glu21Gln	Fluoroquinolones
7,585	gyrA	c.284G>C	p.Ser95Thr	Fluoroquinolones
9,304	gyrA	c.2003G>A	p.Gly668Asp	Fluoroquinolones

What I concluded from each mutation:

The rpoB Asp435Tyr mutation sits in the rifampicin resistance-determining region (RRDR) of the RNA polymerase beta subunit. Rifampicin blocks bacterial RNA polymerase to stop transcription. This mutation alters the drug-binding pocket so rifampicin can no longer bind effectively. About 95% of rifampicin-resistant TB clinical isolates carry mutations in this region, making this a well-established and serious finding.

The katG Ser315Thr mutation was the most significant single finding in my analysis. KatG is the enzyme that activates isoniazid from its inactive prodrug form into its bactericidal form. Without KatG activity, isoniazid never gets activated and cannot kill the bacterium. The Ser315Thr substitution reduces enzyme activity substantially. This specific mutation is the

most common isoniazid resistance mutation globally, found in 50-90% of isoniazid-resistant TB isolates. Its presence in this strain is a strong indicator of isoniazid resistance.

Together, the rpoB and katG mutations confirm MDR-TB: resistance to both rifampicin and isoniazid, the two most important first-line TB drugs. This was my key clinical finding.

The embB Gly406Ser mutation adds ethambutol to the resistance profile. EmbB builds the arabinogalactan layer of the mycobacterial cell wall, and ethambutol inhibits it. This mutation at codon 406 is a known resistance-associated change that reduces drug binding.

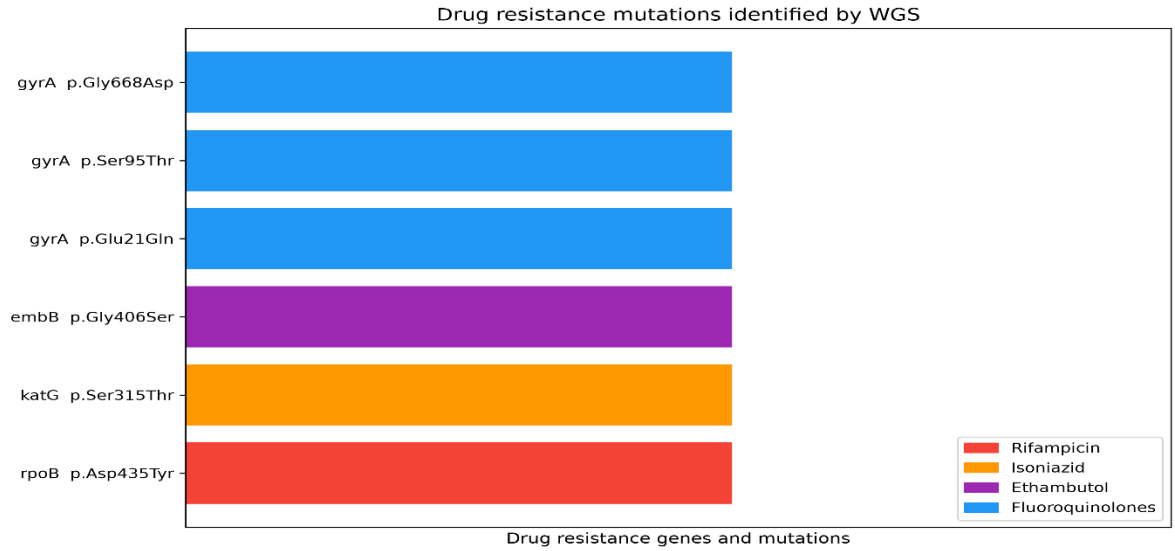


Figure 20: Drug resistance mutations identified by WGS across rpoB, katG, embB, and gyrA, coloured by drug class.

I found three gyrA mutations affecting DNA gyrase subunit A. Fluoroquinolones like moxifloxacin kill TB by trapping the gyrase-DNA complex. These three mutations alter gyrase structure and reduce fluoroquinolone binding. This matters greatly because fluoroquinolones are backbone drugs in MDR-TB treatment regimens. Finding fluoroquinolone resistance in a strain that is already MDR severely limits the treatment options available.

11.4 Overall clinical conclusion

This strain carries resistance to four drug classes: rifampicin, isoniazid, ethambutol and fluoroquinolones. The rpoB and katG mutations confirm MDR-TB by WHO definition. The additional fluoroquinolone resistance classifies this strain as pre-XDR-TB (pre-extensively drug-resistant tuberculosis) under current WHO criteria. This is a clinically serious finding. Pre-XDR-TB requires specialist management, longer treatment courses with more toxic second-line drugs, and significantly worse treatment outcomes than drug-susceptible TB.

Beyond drug resistance, the 51 HIGH impact variants across diverse biological functions, including cell wall biosynthesis, central metabolism and transcriptional regulation, suggest this is a well-adapted clinical strain that has accumulated substantial mutations compared to the H37Rv reference. Some of these changes may contribute to altered virulence or fitness characteristics.